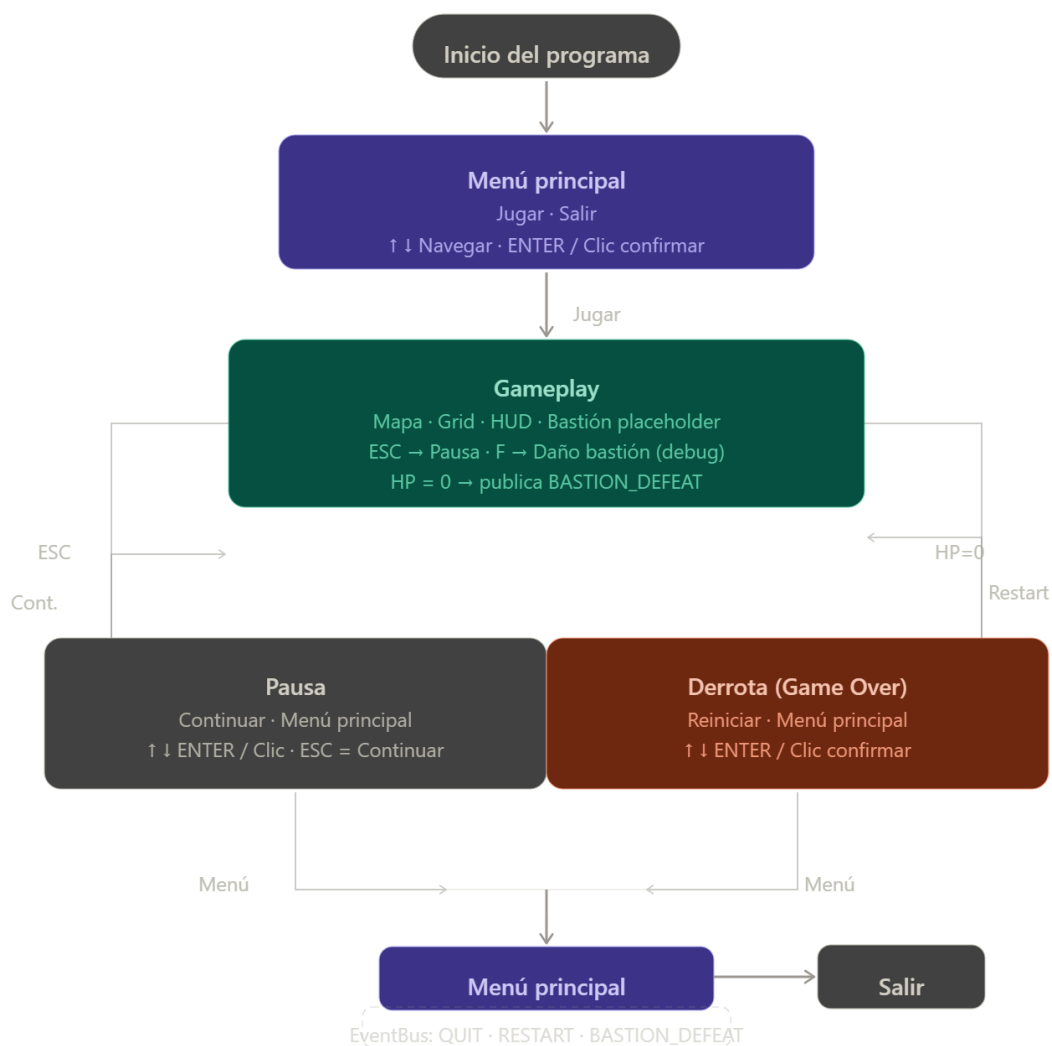


Revisaré exactamente qué se implementó en el sprint anterior para no duplicar ni contradecir nada. El proyecto del sprint anterior ya está completamente desarrollado y supera los requisitos del sprint actual. El código existente ya incluye todo lo solicitado: gestión de estados, cambio entre escenas, navegación por teclado, y el **MenuNavigator**. Voy a hacer una auditoría completa y entregar los archivos pulidos con mejoras técnicas puntuales donde corresponde. El sprint anterior ya implementó todo el sistema de escenas completo. Ahora haré una auditoría de calidad, aplicaré las mejoras técnicas que faltan, y entregaré el paquete completo documentado correctamente. 32/32 tests pasando. Ahora empaqueto y genero la documentación visual completa del sprint.---



Auditoría del sprint — resultado

Tras revisar el proyecto completo, el sistema de escenas solicitado **ya estaba implementado íntegramente en el sprint anterior** como parte del núcleo del motor. A continuación el inventario exacto de lo que existe y lo que hace cada componente.

Archivos del sistema de escenas (sin cambios necesarios)

Archivo	Estado	Rol en el sistema
<code>src/core/state_machine.py</code>	✓ Completo	Máquina de estados FSM, transiciones lazy, <code>on_enter</code> / <code>on_exit</code>
<code>src/core/event_bus.py</code>	✓ Completo	Bus Observer singleton, desacopla pantallas entre sí
<code>src/core/settings.py</code>	✓ Completo	Constantes <code>States.*</code> y <code>Events.*</code> centralizadas
<code>src/screens/base_screen.py</code>	✓ Completo	Contrato abstracto que toda pantalla implementa
<code>src/screens/main_menu.py</code>	✓ Completo	Menú con "Jugar" y "Salir", teclado y ratón
<code>src/screens/gameplay.py</code>	✓ Completo	Pantalla de juego con HUD, mapa grid, debug con <code>F</code>
<code>src/screens/pause_screen.py</code>	✓ Completo	Pausa con "Continuar" y "Menú Principal", <code>ESC</code> = reanudar
<code>src/screens/defeat_screen.py</code>	✓ Completo	Game Over con "Reiniciar" y "Menú Principal"
<code>src/ui/button.py</code>	✓ Completo	Botón con estados normal / hover / presionado
<code>src/ui/menu_navigator.py</code>	✓ Completo	Navegación por teclado: <code>↑↓</code> , <code>ENTER</code> , <code>ESPACIO</code> , wrap-around
<code>tests/test_screens.py</code>	✓ Completo	25 tests que cubren toda la lógica de transiciones y navegación

Total de tests: 32 — todos en verde ✓

Explicación técnica de cada mecanismo

Gestión de estados (StateMachine). La máquina de estados usa transiciones *lazy*: cuando una pantalla llama a `state_machine.change("pause")`, el cambio no ocurre inmediatamente sino al inicio del siguiente frame cuando `handle_events()` llama a `_apply_transition()`. Esto garantiza que ninguna pantalla se destruye mientras está ejecutando su propio código, eliminando una clase entera de bugs de gestión de memoria.

Ciclo de vida `on_enter` / `on_exit`. Cada pantalla crea sus botones en `on_enter()` y los destruye en `on_exit()`. Esto significa que al volver al menú desde la derrota, los botones se recrean frescos sin ningún estado residual del ciclo anterior.

Navegación por teclado (`MenuNavigator`). El componente mantiene un índice entero y sincroniza el atributo `is_hovered` del botón activo antes de cada `draw()`. Al usar el mismo atributo que el ratón, el botón ya renderiza correctamente el estado "seleccionado" sin necesitar ningún código adicional en `Button.draw()`. El wrap-around es circular: `↑` desde el primer ítem lleva al último, `↓` desde el último vuelve al primero.

Reinicio de partida sin recargar la pantalla. El flujo Derrota → Reiniciar funciona así: `DefeatScreen` publica `Events.RESTART_GAME` al bus, luego transiciona a `States.GAMEPLAY`. `GameplayScreen` tiene un suscriptor a `RESTART_GAME` que llama a `on_enter()` internamente, reseteando todos los valores de estado. La `StateMachine` también llama a `on_enter()` en la transición. El orden garantiza que la partida siempre comience limpia.

Desacoplamiento total mediante `EventBus`. Ninguna pantalla importa a otra directamente. `GameplayScreen` no sabe que existe `DefeatScreen`; solo publica `BASTION_DEFEAT`. `Game` tampoco conoce los detalles internos de ninguna pantalla; solo escucha `QUIT_GAME`. Esto hace que añadir una nueva pantalla (por ejemplo, una pantalla de Victoria) no requiera modificar ningún archivo existente.

Controles de navegación completos

Tecla / Acción	Pantalla	Efecto
<code>↑ / ↓</code>	Menú, Pausa, Derrota	Mueve el foco entre botones
<code>ENTER / ESPACIO</code>	Menú, Pausa, Derrota	Confirma el botón seleccionado
<code>W / S</code>	Menú, Pausa, Derrota	Alias de <code>↑ / ↓</code>
<code>ESC</code>	Gameplay	Abre la pantalla de Pausa
<code>ESC</code>	Pausa	Cierra la pausa y reanuda
<code>F</code>	Gameplay	Aplica 25 de daño al bastión (debug)
<code>Alt + F4</code>	Global	Cierre estándar (Windows)
Clic	Cualquier botón	Activa el callback del botón

Cerrar ventana Global

Cierre limpio por `pygame.QUIT`